

End-to-End AQI Prediction System

A Serverless, Automated Pipeline for Rawalpindi

Haroon Rashid

May 24, 2026

Contents

1	Introduction	3
2	System Architecture and Pipeline Design	3
2.1	Data Pipeline	3
2.1.1	Data Source	3
2.1.2	Feature Engineering	4
2.1.3	Automation	4
2.2	Training Pipeline	4
2.2.1	Data Retrieval	4
2.2.2	Model Development	5
2.2.3	Model Registry and Deployment	5
2.3	Inference Pipeline	5
2.3.1	Forecast Generation	5
2.3.2	Prediction Strategy	6
2.4	CI/CD Automation	6
3	Exploratory Data Analysis (EDA) and Feature Insights	6
4	Model Evaluation and SHAP Analysis	7
4.1	Performance Analysis	7
4.2	SHAP Analysis	7
5	Interactive Dashboard: AQI Predictor	7
5.1	Features of the Frontend	8
6	Challenges and Solutions	8
7	Conclusion	8

1 Introduction

Air pollution is a major environmental health issue, particularly in rapidly urbanizing cities like Rawalpindi, Pakistan. Accurate forecasting of the Air Quality Index (AQI) can help residents take preventive measures and assist policymakers in implementing timely interventions.

This project presents the design and implementation of a **100% serverless, end-to-end AQI prediction system**. The system automates the entire machine learning lifecycle: from hourly data ingestion and feature engineering to model training, deployment, and interactive visualization. It predicts the AQI for the next 3 days with hourly granularity, providing a valuable tool for public health awareness.

2 System Architecture and Pipeline Design

The architecture follows a modern data-centric approach, leveraging a feature store and CI/CD automation to ensure reliability and scalability.

Stage	Description	Technology
Data Ingestion	Fetches weather & pollutant data hourly	GitHub Actions, Open-Meteo
Feature Store	Stores engineered features and targets	Hopsworks
Model Training	Daily retraining with hyperparameter tuning	Scikit-learn, XGBoost, Hopsworks
Model Registry	Versioning and deployment of models	Hopsworks
Inference	Generates 3-day (max 5-day) daily and hourly forecasts	Deployed models, Streamlit
CI/CD	Automation of pipelines	GitHub Actions

Figure 1: System Component Overview

2.1 Data Pipeline

The data pipeline is responsible for fetching raw data, engineering features, and storing them in the Hopsworks Feature Store.

2.1.1 Data Source

The system utilizes the **Open-Meteo API**, which provides:

- **Weather Features:** Temperature, humidity, wind speed, pressure, precipitation, etc.
- **Pollutant Data:** PM10, PM2.5, CO, NO2, SO2, Ozone.
- **Target Variable:** US AQI (`us_aqi`).

Coordinates: Rawalpindi (Latitude: 33.5973, Longitude: 73.0479).

2.1.2 Feature Engineering

To improve model performance, temporal and domain-specific features are created:

- **Temporal Features:** Hour, day of week, month, day of year, weekend flag, rush hour flag.
- **Season Encoding:** Winter (Dec-Feb) is mapped to 0 to capture smog effects. Spring is mapped to 1, Summer to 2, and Autumn to 3.
- **Wind Vector Decomposition:** Wind direction is decomposed into U (east-west) and V (north-south) components.
- **Derived Features:** Stagnation flag (wind speed < 2 m/s), temperature-humidity product.

2.1.3 Automation

A GitHub Actions workflow (`data_pipeline.yml`) runs the `dataset_pipeline.py` script every hour (cron: `5 * * * *`). This ensures the feature store is continuously updated with the latest hourly data, using the `datetime` column as the primary key to avoid duplicates.

The historical data backfill is performed only once, manually, populating the feature store with 3 months of past data.

2.2 Training Pipeline

The training pipeline is designed for daily model retraining to adapt to evolving pollution patterns.

2.2.1 Data Retrieval

The pipeline fetches the stored data from the feature group `aqi_hourly_features` in Hopsworks. The data is split into train (90%) and test (10%) sets.

2.2.2 Model Development

Five models were selected to capture different aspects of the data:

1. **Random Forest Regressor**
2. **Gradient Boosting Regressor**
3. **Support Vector Regressor (SVR)** with linear kernel
4. **K-Nearest Neighbors (KNN)** Regressor
5. **XGBoost Regressor**

All models are trained using standard hyperparameters (`n_estimators=100`, `random_state=42`) and parallel processing where available.

2.2.3 Model Registry and Deployment

A key implementation detail is the management of model versions. Since Hopsworks increments versions automatically, making it hard to track the latest model, the pipeline implements a strategy to:

1. Stop any existing deployment for the model.
2. Delete the previous version of the model.
3. Save the newly trained model with `version=1`.
4. Deploy the model using KSERVE. For XGBoost, a custom predictor script is required.

This approach ensures that only the latest, best-performing model is active, saving storage and simplifying inference.

2.3 Inference Pipeline

The inference pipeline is designed to be called by the frontend application on-demand.

2.3.1 Forecast Generation

It fetches forecast data for weather and pollutants for the next 3 days (configurable up to 5 days). It then applies the exact same feature engineering logic used during training to the forecast data. This consistency is crucial for model accuracy.

2.3.2 Prediction Strategy

Research was carried out on the topic of daily AQI prediction. The results showed that the US AQI (the target) is not predicted as a daily variable, rather it makes use of hourly AQI values for the prediction of daily AQI. Keeping this in mind instead of training separate models for daily AQI, the models predict hourly AQI. The final daily AQI is then calculated as the mean of the hourly predictions for that day. This strategy allows the model to be more robust and capture intra-day pollution spikes.

2.4 CI/CD Automation

The entire system is automated using GitHub Actions:

- **Data Pipeline** (`data_pipeline.yml`): Triggers every hour to fetch and store new data.
- **Training Pipeline** (`training_pipeline.yml`): Triggers daily to retrain and deploy models.

Secrets like the Hopsworks API key are securely stored in GitHub Secrets.

3 Exploratory Data Analysis (EDA) and Feature Insights

An EDA was conducted to understand the data before modeling. Key findings include:

- **Target Distribution:** The US AQI in the dataset primarily falls in the "Moderate" (0-50) and "Unhealthy for Sensitive Groups" (101-150) categories. No "Good" or "Hazardous" levels were observed in this sample.
- **Feature Correlations:** As expected, PM2.5, PM10, and SO2 show the strongest positive correlations with AQI. Temperature also has a moderate positive correlation, while relative humidity shows a negative correlation.
- **Temporal Patterns:** AQI shows a distinct diurnal pattern, with lower values in the afternoon and higher values in the evening, likely due to increased vehicular emissions and atmospheric conditions.
- **Seasonal Trends:** Winter months (December-February) exhibit higher AQI levels, confirming the impact of temperature inversions that trap pollutants.

These insights justified the inclusion of engineered temporal and seasonal features.

4 Model Evaluation and SHAP Analysis

The models were evaluated on a held-out test set (10% of the data). The primary metrics used were R-squared (R^2) and Mean Absolute Error (MAE).

Table 1: Model Performance Metrics

Model	R^2 Score	MAE (AQI)
Random Forest	0.981	2.13
Gradient Boosting	0.903	6.61
SVR	0.570	13.28
KNN	0.240	18.25
XGBoost	0.982	1.30

4.1 Performance Analysis

The tree-based ensemble methods (Random Forest and XGBoost) significantly outperformed the other models. XGBoost achieved the best results with an R^2 of 0.982 and an MAE of just 1.30 AQI points. This high performance indicates that the engineered features are highly predictive of AQI. The poor performance of SVR and KNN suggests that the relationship, while strong, is non-linear and complex.

4.2 SHAP Analysis

SHAP (SHapley Additive exPlanations) was used to interpret the best-performing XGBoost model. The SHAP summary plot reveals:

- **PM2.5** and **PM10** are the most important features, with high values pushing the AQI higher.
- **Other pollutants** also play a significant role; each of the pollutants has a clear separation of high and low values, this shows that they have a linear relation with AQI.

The SHAP analysis provides transparency, showing that the model's predictions are based on physically meaningful features.

5 Interactive Dashboard: AQI Predictor

An interactive web application was built using **Streamlit** to showcase the predictions.

5.1 Features of the Frontend

- **Model Selection:** Users can view predictions from any of the five individual models or the ensemble mean.
- **Hourly Forecast Visualization:** An interactive Plotly chart displays the 72-hour AQI forecast.
- **AQI Category Coloring:** The chart background is shaded according to AQI categories (Good, Moderate, Unhealthy, etc.) for quick health risk assessment.
- **Daily Summary Cards:** The predicted AQI for each day is presented in a clean, card-based layout.
- **Real-time Updates:** Predictions are generated on-demand by calling the inference pipeline, ensuring fresh forecasts.

The dashboard serves as the public-facing component of the system, making complex model outputs easily understandable.

6 Challenges and Solutions

Several technical challenges were encountered during the project:

- **Limited Historical Data:** Open-Meteo provides only 92 days of historical data. **Solution:** The project utilizes hourly data gathering to accumulate more data over time for better long-term trends.
- **Hopworks Model Versioning:** Automatic version incrementing made it difficult to fetch the latest model version. **Solution:** Implemented a "delete and redeploy" strategy in the training pipeline. This ensures only version 1 exists, simplifying the inference pipeline's logic to always fetch version 1.
- **XGBoost Deployment on Hopworks:** Deploying XGBoost required a custom inference script. **Solution:** A custom predictor script (`xgboost_predictor.py`) was written and uploaded to Hopworks resources. The deployment call was modified to reference this script.

7 Conclusion

This project successfully delivers a fully functional, automated, and serverless AQI prediction system for Rawalpindi. The system fulfills all the specified requirements:

1. **End-to-End System:** Data ingestion, feature engineering, training, deployment, and visualization are integrated.
2. **Scalable, Automated Pipelines:** GitHub Actions and Hopsworks provide a robust, serverless foundation, with hourly data updates and daily model retraining.
3. **Interactive Dashboard:** A Streamlit application provides real-time forecasts with model selection and visual aids.
4. **Comprehensive Documentation:** This report details the architecture, implementation, and results.

The XGBoost model achieved excellent predictive performance ($R^2 = 0.982$), demonstrating the effectiveness of the feature engineering strategy. While limited to 92 days of data, the system is designed for continuous operation and will only improve as more data is collected. Future work could include adding alert systems for hazardous AQI levels and exploring deep learning models like LSTMs for time-series forecasting.